Code Explanation

Test Batch Processing Code Explanation

This is a unit test class written in Python using the unittest framework to test the functionality of batch processing code. The batch processing code is designed to clean customer and order data using Apache Spark.

Overview
The provided code is a test suite for batch processing functionality. It creates a Spark session, defines test data, and tests the cleaning of customer and order data. The test cases verify that the cleaning process removes null values and duplicates, and that the calculated TotalAmount is correct.

Step 1: Setting Up the Test Environment
This step involves creating a Spark session and defining the test data for customer and order information. The test data includes duplicate records and null values to test the cleaning functionality.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestBatchProcessing")
        .master("local[*]")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSess
ionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark
.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Defining Test Data
This step defines the test data for customers and orders, including schema definitions for the DataFrames.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "East"),
    ("C002", "Jane Smith", "jane@example.com", "West"),
    ("C003", None, "bob@example.com", "North"),
    ("C004", "Alice Brown", None, "South"),
    ("C001", "John Doe", "john@example.com", "East")  # Duplicate
]

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_df = cls.spark.createDataFrame(customer_data, schema=cu
stomer_schema)
```

Step 3: Cleaning Customer Data

This step tests the `clean_customer_data` function by verifying that it removes null values and duplicates.

```
def test_clean_customer_data(self):
    cleaned_customer = clean_customer_data(self.customer_df)
    self.assertEqual(cleaned_customer.count(), 2)
    null_counts = cleaned_customer.select([
        sum(col.isNull().cast("int")).alias(col_name)
        for col_name, col in cleaned_customer.dtypes
    ]).collect()[0]
    for count in null_counts:
        self.assertEqual(count, 0)
```

Step 4: Cleaning Order Data
This step tests the `clean_order_data` function by verifying that it removes null values and duplicates, and that the calculated TotalAmount is correct.

```
def test_clean_order_data(self):
    cleaned_order = clean_order_data(self.order_df)
    self.assertEqual(cleaned_order.count(), 2)
    order1 = cleaned_order.filter("OrderId = 'O001'").collect()[0]
    self.assertEqual(order1["TotalAmount"], 20.0)  # 10.0 * 2
    order2 = cleaned_order.filter("OrderId = 'O002'").collect()[0]
    self.assertEqual(order2["TotalAmount"], 15.0)  # 15.0 * 1
```

Step 5: Tearing Down the Test Environment
This step involves stopping the Spark session after the tests are completed.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
```